

# Important Exam Preparation Notes (According to Ma'am Suggestion + Slides)

---

## 1. Greedy Techniques

### Important Theory

#### Greedy Algorithm

A greedy algorithm chooses the best available option at each step without considering future consequences. It is mainly used for optimization problems.

#### Properties of Greedy Algorithm

##### Greedy Choice Property

Optimal solution can be obtained by making locally optimal choices.

##### Optimal Substructure

Optimal solution contains optimal solutions of subproblems.

#### Limitation

Greedy algorithms do not always provide optimal solutions. Example:

- Coin Change Problem
- 0/1 Knapsack Problem

#### Fractional Knapsack

Items can be taken fractionally to maximize total profit. Best strategy: Profit/Weight Ratio.

---

## Important Short Questions

### Q1. What is a greedy algorithm?

A greedy algorithm selects the best option at each step to solve optimization problems.

## Q2. What is Greedy Choice Property?

It means making local optimal choices leads to global optimal solution.

## Q3. What is Optimal Substructure?

Optimal solution contains optimal solutions of subproblems.

## Q4. Why does greedy fail sometimes?

Because local optimum may not produce global optimum.

## Q5. Which strategy is best in Fractional Knapsack?

Profit/Weight ratio.

---

# Important MCQ

## Q1. Greedy algorithms are mainly used for —

- A) Traversal
- B) Optimization
- C) Sorting
- D) Searching

**Answer:** B) Optimization

---

## Q2. Which property is necessary for greedy algorithms?

- A) BFS Property
- B) DFS Property
- C) Greedy Choice Property
- D) Coloring Property

**Answer:** C) Greedy Choice Property

---

## Q3. Fractional Knapsack uses —

- A) BFS
- B) Greedy

C) DFS

D) DP

**Answer:** B) Greedy

---

## 2. Single Source Shortest Path (SSSP)

### Important Theory

#### SSSP

Finding shortest paths from one source node to all other nodes.

#### Dijkstra's Algorithm

- Greedy algorithm
- Works on weighted graphs
- Does not work with negative edges

#### Relaxation

$$d(u) + c(u, v) < d(v)$$

Then:

$$d(v) = d(u) + c(u, v)$$

#### Bellman-Ford Algorithm

- Works with negative edges
  - Relaxes all edges  $V-1$  times
  - Detects negative cycles
- 

## Important Short Questions

### Q1. What is SSSP?

Finding shortest path from one source node to all nodes.

## Q2. Which algorithm does not work with negative edges?

Dijkstra's Algorithm.

## Q3. Which algorithm detects negative cycles?

Bellman-Ford Algorithm.

## Q4. What is relaxation?

Updating shorter distance if found.

## Q5. How many times are edges relaxed in Bellman-Ford?

$V-1$  times.

---

# Important MCQ

## Q1. Dijkstra's Algorithm follows —

- A) DP
- B) BFS
- C) Greedy
- D) Backtracking

**Answer:** C) Greedy

---

## Q2. Bellman-Ford works with —

- A) Negative edges
- B) Only positive edges
- C) Trees only
- D) DAG only

**Answer:** A) Negative edges

---

## Q3. Initial distance of source node in Dijkstra is —

- A)  $\infty$
- B) 1

- C) 0
- D) -1

**Answer:** C) 0

---

## Important Problems

### Problem Type 1: Dijkstra Algorithm Solve

Steps:

1. Initialize source = 0
  2. Others = INF
  3. Select minimum distance node
  4. Relax adjacent nodes
  5. Repeat
- 

### Problem Type 2: Bellman-Ford Solve

Steps:

1. Initialize distances
  2. Relax all edges  $V-1$  times
  3. Check negative cycle
- 

## 3. Minimum Spanning Tree (MST)

## Important Theory

### Spanning Tree

A subgraph containing all vertices and forms a tree.

### Minimum Spanning Tree

Spanning tree with minimum total cost.

## Kruskal's Algorithm

- Greedy method
  - Select minimum edge first
  - Avoid cycles
- 

## Important Short Questions

### Q1. What is MST?

Minimum cost spanning tree.

### Q2. Name MST algorithms.

- Kruskal
- Prim

### Q3. What happens if cycle forms in Kruskal?

Edge is rejected.

### Q4. How many edges are in MST?

$n-1$  edges.

---

## Important MCQ

### Q1. MST stands for —

- A) Minimum Sorting Tree
- B) Minimum Spanning Tree
- C) Maximum Spanning Tree
- D) Minimum Search Tree

**Answer:** B) Minimum Spanning Tree

---

## Q2. Kruskal's Algorithm selects —

- A) Maximum edge
- B) Random edge
- C) Minimum edge
- D) Negative edge

**Answer:** C) Minimum edge

---

## Q3. MST of a graph with $n$ vertices contains —

- A)  $n$
- B)  $n-1$
- C)  $n+1$
- D)  $n^2$

**Answer:** B)  $n-1$

---

# Important Problems

## Problem Type: Kruskal's Algorithm

Steps:

1. Sort edges
  2. Pick smallest edge
  3. Avoid cycles
  4. Continue until  $n-1$  edges selected
- 

## 4. Backtracking

## Important Theory

### Backtracking

A technique where solutions are built incrementally and invalid solutions are abandoned.

## Steps

- Choose
- Explore
- Check validity
- Backtrack

## Applications

- N Queens
  - Graph Coloring
  - Sudoku
- 

# Important Short Questions

## Q1. What is backtracking?

A recursive problem-solving technique.

## Q2. What happens at dead end?

Algorithm returns to previous state.

## Q3. Name applications of backtracking.

- N Queens
  - Graph Coloring
- 

# Important MCQ

## Q1. N Queens Problem uses —

- A) Greedy
- B) BFS
- C) Backtracking
- D) DP

**Answer:** C) Backtracking



---

## Q2. Graph Coloring ensures adjacent vertices have —

- A) Same color
- B) Different colors
- C) Same weight
- D) Same edge

**Answer:** B) Different colors

---

## Important Problem

### Problem Type: N Queens

Need to place queens so that:

- No same row
  - No same column
  - No same diagonal
- 

## 5. Topological Sort

## Important Theory

### Topological Sorting

Linear ordering of vertices in DAG where:

$u \rightarrow v$  means  $u$  appears before  $v$ .

### Kahn's Algorithm

- BFS based
- Uses indegree
- Queue based

### DFS Topological Sort

Uses stack after DFS traversal.

---

## Important Short Questions

**Q1. Topological sort is possible only for which graph?**

DAG.

**Q2. What is indegree?**

Number of incoming edges.

**Q3. Which algorithm uses queue?**

Kahn's Algorithm.

---

## Important MCQ

**Q1. Topological sorting is possible only for —**

- A) Cyclic Graph
- B) DAG
- C) Tree
- D) Undirected Graph

**Answer:** B) DAG

---

**Q2. Kahn's Algorithm is based on —**

- A) DFS
- B) DP
- C) BFS
- D) Greedy

**Answer:** C) BFS

---

# Important Problem

## Problem Type: Topological Sort

1. Find indegree
  2. Push indegree 0 nodes
  3. Remove node
  4. Reduce indegree
  5. Repeat
- 

## 6. Branch and Bound

### Important Theory

#### Branch and Bound

Optimization technique using:

- Branching
- Bounding
- Pruning

#### FIFO Branch and Bound

Uses queue and BFS.

---

## Important Short Questions

### Q1. What is pruning?

Removing unnecessary branches.

### Q2. What is branching?

Dividing problem into subproblems.

### Q3. Which search is used in FIFO Branch and Bound?

BFS.

---

## Important MCQ

### Q1. Branch and Bound is used for —

- A) Optimization
- B) Traversal
- C) Searching
- D) Sorting

**Answer:** A) Optimization

---

### Q2. FIFO Branch and Bound uses —

- A) Stack
- B) Queue
- C) Heap
- D) Tree

**Answer:** B) Queue

---

## 7. Approximation Algorithm

### Important Theory

#### Approximation Algorithm

Algorithms that provide near-optimal solutions for NP-hard problems.

#### TSP

Visit all cities exactly once and return to source with minimum cost.

#### Triangle Inequality

$$d(A,C) \leq d(A,B) + d(B,C)$$

## **Vertex Cover**

Subset of vertices covering all edges.

---

## **Important Short Questions**

### **Q1. What is approximation algorithm?**

Near-optimal solution algorithm.

### **Q2. What type of problem is TSP?**

NP-hard.

### **Q3. What is Vertex Cover?**

Subset of vertices covering all edges.

---

## **Important MCQ**

### **Q1. TSP stands for —**

- A) Tree Search Problem
- B) Traveling Salesman Problem
- C) Topological Search Problem
- D) Traversal Source Problem

**Answer:** B) Traveling Salesman Problem

---

### **Q2. Approximation algorithms are used for —**

- A) Easy problems
- B) NP-hard problems
- C) Trees only
- D) Searching only

**Answer:** B) NP-hard problems

---

### **Q3. Vertex Cover covers —**

- A) Vertices only
- B) Cycles
- C) All edges
- D) Paths only

**Answer:** C) All edges

---

## **Important Problems**

### **Problem Type 1: TSP Approximation**

1. Build MST
2. Preorder traversal
3. Construct path

### **Problem Type 2: Vertex Cover**

1. Select edge
2. Add endpoints
3. Remove incident edges
4. Repeat until covered